

LAB ASSIGNMENT – 9.5

SHREE PRIYA

2303A51465

BATCH – 29

Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):  
    return text[::-1]
```

Task:

1. Write documentation in:
 - o (a) Docstring
 - o (b) Inline comments
 - o (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string

Library

PROMPT: Write Python documentation for a string utility function using docstring, inline comments, and Google-style format, compare them, and recommend the best style

CODE:

```
def reverse_string(text):  
    return text[::-1]  
  
# Docstring style  
def reverse_string_docstring(text):  
    """  
    Reverses the given string.  
    Args:  
        text (str): The string to be reversed.  
    Returns:  
        str: The reversed string.  
    """  
    return text[::-1]
```

Inline comments style

```
def reverse_string_inline(text):  
    # Reverse the string using slicing  
    return text[::-1]
```

Google-style format

```
def reverse_string_google(text):
```

```
    """Reverses the given string.
```

```
    Args:
```

```
        text: The string to be reversed.
```

```
    Returns:
```

```
        The reversed string.
```

```
    """
```

```
    return text[::-1]
```

Comparison and recommendation

The docstring style provides a clear and structured way to document the function, including the purpose, arguments, and return value. It is widely used in Python and is recommended for its readability and consistency.

The inline comments style is less formal and may not provide as much information about the function's purpose and usage. It can be useful for simple functions but may not be sufficient for more complex ones.

The Google-style format is similar to the docstring style but follows specific formatting guidelines. It can be beneficial for teams that follow Google's coding standards, but it may be less familiar to developers who are used to the traditional docstring style.

Overall, the docstring style is recommended for its clarity and widespread use in the Python community. It provides a comprehensive way to document the function while maintaining readability and consistency.

Justification :

Docstring and Google-style documentation clearly explain parameters and return values. Inline comments are simple but not structured. For a utility-based string library, Google-style is most suitable because it is clean, structured, and widely used in professional projects. It improves readability and maintainability.

```

1 # Write Python documentation for a string utility function using docstring, inline comments, and Google-style format, compare them, and recommend the best style
2 def reverse_string(text):
3     return text[::-1]
4
5 # Docstring style
6 def reverse_string_docstring(text):
7     """
8     Reverse the given string.
9
10    Args:
11        text (str): The string to be reversed.
12
13    Returns:
14        str: The reversed string.
15    """
16     return text[::-1]
17
18 # Inline comments style
19 def reverse_string_inline(text):
20     # Reverse the string using slicing
21     return text[::-1]
22
23 # Google-style format
24 def reverse_string_google(text):
25     """Reverse the given string.
26
27     Args:
28         text: The string to be reversed.
29
30     Returns:
31         The reversed string.
32     """
33     return text[::-1]
34
35 # Comparison and recommendation
36 # The docstring style provides a clear and structured way to document the function, including its purpose, arguments, and return value. It is widely used in Python and is recommended for its readability and
37 # The inline comments style is less formal and may not provide as much information about the function's purpose and usage. It can be useful for simple functions but may not be sufficient for more complex ones.
38 # The Google-style format is similar to the docstring style but follows specific formatting guidelines. It can be beneficial for teams that follow Google's coding standards, but it may be less familiar to others.
39 # Overall, the docstring style is recommended for its clarity and widespread use in the Python community. It provides a comprehensive way to document the function while maintaining readability and consistency.

```

Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):
    return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

PROMPT: # Write documentation for a Python password strength checker function using basic docstring, inline comments, and Google-style format, then compare the styles and recommend the most suitable one for security-related code.

CODE:

```
def check_strength(password):
    return len(password) >= 8

# Basic docstring style
def check_strength_docstring(password):
    """
    Checks the strength of a password.

```

Parameters:

password (str): The password to be checked.

Returns:

bool: True if the password length is at least 8 characters,
False otherwise.

"""

return len(password) >= 8

Inline comments style

def check_strength_inline(password):

Returns True if password length is >= 8

return len(password) >= 8

Google-style format

def check_strength_google(password):

"""Checks whether a password meets minimum length requirement.

Args:

password (str): Password string to validate.

Returns:

bool: True if password length >= 8, otherwise False.

"""

return len(password) >= 8

Comparison and recommendation

For security-related code, the docstring style is recommended as it provides a clear and structured

way to document the function, including the purpose, arguments, and return value. It allows developers to understand the function's intent and usage easily, which is crucial for security-related code.

The inline comments style may not provide enough information about the function's purpose and usage, which can lead to misunderstandings or misuse of the function in a security context.

The Google-style format is similar to the docstring style but may be less familiar to developers who are used to the traditional docstring style. It can be beneficial for teams that follow Google's coding standards, but it may not be the best choice for security-related code if the team is not familiar with it.

Overall, the docstring style is recommended for security-related code due to its clarity, comprehensiveness, and widespread use in the Python community. It helps ensure that developers understand the function's purpose and usage, which is essential for maintaining security.

Example usage

```
pwd = input("Enter password: ")  
  
print(check_strength(pwd))
```

Justification :

For security-related code, documentation must clearly describe input, output, and validation rules. Inline comments are not sufficient for explaining security logic. Google-style documentation is best because it clearly separates arguments, returns, and behavior, making it easier for auditing and team development.

```
# Python documentation for a Python password checker function using basic docstring, inline comments, and Google-style format. This example illustrates the style and structure that should be followed for security-related code.  
  
def check_strength(password):  
    """Returns True if password length is >= 8, otherwise False.  
    Parameters:  
        password (str): The password to be checked.  
    Returns:  
        bool: True if the password length is at least 8 characters,  
            False otherwise.  
    """  
    return len(password) >= 8  
  
# Google-style format  
def check_strength_google(password):  
    """Returns True if password length is >= 8, otherwise False.  
    Parameters:  
        password (str): Password string to validate.  
    Returns:  
        bool: True if password length >= 8, otherwise False.  
    """  
    return len(password) >= 8  
  
# Comparison and recommendation  
# For security-related code, the Google-style is recommended as it provides a clear and structured  
# way to document the function, including the purpose, arguments, and return value. It allows developers to understand the function's intent and usage easily, which is crucial for security-related code.  
# The inline comments style may not provide enough information about the function's purpose and usage, which can lead to misunderstandings or misuse of the function in a security context.  
# The Google-style format is similar to the docstring style but may be less familiar to developers who are used to the traditional docstring style. It can be beneficial to train team members on Google's coding  
# standards, the docstring style is recommended for security-related code due to its clarity, consistency, and widespread use in the Python community. It helps ensure that developers understand the function's  
# purpose and usage.  
pwd = input("Enter password: ")  
print(check_strength(pwd))
```

Problem 3: Math Utilities Module

Task:

1. Create a module `math_utils.py` with functions:
 - o `square(n)`
 - o `cube(n)`
 - o `factorial(n)`
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

PROMPT: # Create a Python module `math_utils.py` containing `square(n)`, `cube(n)`, and `factorial(n)` functions, auto-generate docstrings using an AI tool, and export the module documentation as an HTML file

CODE:

```
# math_utils.py  
"""
```

math_utils.py

A simple math utilities module providing square, cube, and factorial functions.

```
"""
```

```
def square(n):
```

```
    """
```

```
    Calculate the square of a number.
```

```
    Parameters:
```

```
        n (int or float): The number to be squared.
```

```
    Returns:
```

```
        int or float: The square of the number.
```

```
    """
```

```
    return n * n
```

```
def cube(n):
```

```
    """
```

```
    Calculate the cube of a number.
```

```
    Parameters:
```

```
        n (int or float): The number to be cubed.
```

```
    Returns:
```

```
        int or float: The cube of the number.
```

```
    """
```

```
    return n * n * n
```

```
def factorial(n):
```

```
    """
```

```
    Calculate the factorial of a non-negative integer.
```

```
    Parameters:
```

```
        n (int): Non-negative integer.
```

```
    Returns:
```

```
        int: Factorial of n.
```

```
    Raises:
```

```
        ValueError: If n is negative.
```

```
    """
```

```

if n < 0:

    raise ValueError("Factorial is not defined for negative numbers.")

if n == 0 or n == 1:

    return 1

result = 1

for i in range(2, n + 1):

    result *= i

return result

```

Justification :

Mathematical utility modules are reusable components, so clear documentation is important. Proper docstrings help users understand function usage and return types. Generating HTML documentation improves accessibility. Structured documentation ensures better maintenance and scalability.

```

C:\Users\j\Downloads> python math_utils.py
1  # Create a Python module 'math_utils.py' containing square(x), cube(x), and factorial(n) functions, auto-generate docstrings using an AI tool, and export the module documentation as an HTML file
2  # math_utils.py
3  """
4  math_utils.py
5
6  A simple math utilities module providing square, cube, and factorial functions.
7  """
8
9  def square(x):
10     """
11     Calculate the square of a number.
12
13     Parameters:
14         x (int or float): The number to be squared.
15
16     Returns:
17         int or float: The square of the number.
18     """
19     return x * x
20
21 def cube(x):
22     """
23     Calculate the cube of a number.
24
25     Parameters:
26         x (int or float): The number to be cubed.
27
28     Returns:
29         int or float: The cube of the number.
30     """
31     return x * x * x
32
33 def factorial(x):
34     """
35     Calculate the factorial of a non-negative integer.
36
37     Parameters:
38         x (int): Non-negative integer.
39
40     Returns:
41         int: Factorial of x.
42
43     Raises:
44         ValueError: If x is negative.
45     """
46     if x < 0:
47         raise ValueError("Factorial is not defined for negative numbers.")
48     if x == 0 or x == 1:
49         return 1
50     result = 1
51     for i in range(2, x + 1):
52         result *= i
53     return result
54
55 # Module documentation
56 """
57 Module: math_utils
58 """

```

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:

- o mark_present(student)
- o mark_absent(student)

o get_attendance(student)

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

PROMPT: Create a Python module `attendance.py` with functions `mark\_present(student)`, `mark\_absent(student)`, and `get\_attendance(student)`, add proper docstrings, and generate/view its documentation in the terminal and browser using pydoc.

CODE:

```
# attendance.py
```

```
"""
```

```
attendance.py
```

```
A simple attendance management module for tracking student presence.
```

```
"""
```

```
# Dictionary to store attendance records
```

```
_attendance_record = {}
```

```
def mark_present(student):
```

```
    """
```

```
    Marks a student as present.
```

```
    Args:
```

```
        student (str): Name of the student.
```

```
    Returns:
```

```
        None
```

```
    """
```

```
    _attendance_record[student] = "Present"
```

```
def mark_absent(student):
```

```
    """
```

```
    Marks a student as absent.
```

```
    Args:
```

```
        student (str): Name of the student.
```

```
    Returns:
```

```
        None
```

```
    """
```



```
_attendance_record[student] = "Absent"

def get_attendance(student):
    """
    Retrieves the attendance status of a student.

    Args:
        student (str): Name of the student.

    Returns:
        str: Attendance status ("Present" or "Absent").
        Returns "No record found" if student does not exist.
    """
    return _attendance_record.get(student, "No record found")
```

Justification:

Attendance systems involve data handling, so clear function documentation is necessary. Docstrings help explain purpose, inputs, and outputs. Viewing documentation in terminal and browser improves usability. Structured documentation ensures clarity in multi-function modules.

```
4 attendance.py
5 A simple attendance management module for tracking student presence.
6 """
7 # Dictionary to store attendance records
8 _attendance_record = {}
9 def mark_present(student):
10     """
11     Marks a student as present.
12
13     Args:
14         student (str): Name of the student.
15
16     Returns:
17         None
18     """
19     _attendance_record[student] = "Present"
20 def mark_absent(student):
21     """
22     Marks a student as absent.
23
24     Args:
25         student (str): Name of the student.
26
27     Returns:
28         None
29     """
30     _attendance_record[student] = "Absent"
31 def get_attendance(student):
32     """
33     Retrieves the attendance status of a student.
34
35     Args:
36         student (str): Name of the student.
37
38     Returns:
39         str: Attendance status ("Present" or "Absent").
40         Returns "No record found" if student does not exist.
41     """
42     return _attendance_record.get(student, "No record found")
43
```

PROBLEMS OUTPUT TERMINAL PORTS

> TERMINAL

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> c:: cd 'C:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\Windows
0-win32-x64\bin\debugpy\launcher' '63218' -c 'C:\Users\Shree priya\Downloads\attendance.py'
PS C:\Users\Shree priya\Downloads> python -m pydoc -w attendance
wrote attendance.html
PS C:\Users\Shree priya\Downloads>
```

Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):  
  
    with open(filename, 'r') as f:  
  
    return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation

PROMPT : Write documentation for a Python file-reading function using basic docstring, inline comments, and Google-style format, then compare the styles to determine which best explains exception handling and justify the recommendation.

CODE:

Original function

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()
```

Basic Docstring Style

```
def read_file_docstring(filename):  
    """  
  
    Reads the contents of a file.  
  
    Parameters:  
  
        filename (str): The file name or file path.  
  
    Returns:  
  
        str: The contents of the file.  
  
    Raises:  
  
        FileNotFoundError: If the file does not exist.  
    """  
  
    with open(filename, 'r') as f:  
        return f.read()
```

Inline Comments Style

```
def read_file_inline(filename):  
    # Open the file in read mode  
    # Read all contents  
    # Return file content  
    with open(filename, 'r') as f:  
        return f.read()
```

Google-Style Documentation

```
def read_file_google(filename):  
    """Reads and returns the contents of a file.  
  
    Args:  
  
        filename (str): Path to the file.  
  
    Returns:  
  
        str: File contents.
```

Raises:

FileNotFoundError: If file is not found.

OSError: If another I/O error occurs.

"""

with open(filename, 'r') as f:

return f.read()

---- MAIN EXECUTION ----

if __name__ == "__main__":

try:

content = read_file("data.txt")

print("File Content:\n")

print(content)

except FileNotFoundError:

print("Error: File not found. Please create data.txt.")

Justification :

File handling functions may raise exceptions like FileNotFoundError. Therefore, documentation must clearly mention possible errors. Google-style documentation is best because it includes a dedicated “Raises” section. This improves clarity, especially for exception handling and debugging.

```

C:\Users\Shree priya\Downloads> File reader.py
1 # Write documentation for a Python file-reading function using basic docstring, inline comments, and Google-style format, then compare the styles
2 # Original function
3 def read_file(filename):
4     with open(filename, 'r') as f:
5         return f.read()
6 # Basic Docstring Style
7 def read_file_docstring(filename):
8     """
9     Reads the contents of a file.
10    Parameters:
11        filename (str): The file name or file path.
12    Returns:
13        str: The contents of the file.
14    Raises:
15        FileNotFoundError: If the file does not exist.
16    """
17    with open(filename, 'r') as f:
18        return f.read()
19 # Inline Comments Style
20 def read_file_inline(filename):
21     # Open the file in read mode
22     # Read all contents
23     # Return file content
24     with open(filename, 'r') as f:
25         return f.read()
26 # Google-Style Documentation
27 def read_file_google(filename):
28     """Reads and returns the contents of a file.
29
30     Args:
31         filename (str): Path to the file.
32
33     Returns:
34         str: File contents.
35
36     Raises:
37         FileNotFoundError: If file is not found.
38         IOError: If another I/O error occurs.
39     """
40     with open(filename, 'r') as f:
41         return f.read()
42 # ---- MAIN EXECUTION ----
43 if __name__ == "__main__":
44     try:
45         content = read_file("data.txt")
46         print("File Content:\n")
47         print(content)
48     except FileNotFoundError:
49         print("Error: File not found. Please create data.txt.")

```

PROBLEMS OUTPUT TERMINAL PORTS

~ TERMINAL

```

Hello Priya
Python file handling working!
PS C:\Users\Shree priya\Downloads>

```